NimbusHR Engineering Guidelines

Version 3.1 | Last Updated: January 2025

---

1. Engineering Principles

1.1 Core Philosophy

NimbusHR values **simplicity, maintainability, and testability** in all software systems. We believe that well-crafted code is:

- **Simple**: Easy to understand and reason about
- **Maintainable**: Can be modified and extended without breaking existing functionality
- **Testable**: Designed with automated testing in mind from the start
- **Scalable**: Capable of handling growth in users, data, and features
- **Secure**: Built with security best practices at every layer

1.2 Code Quality Standards

Write for Readability:

- Use descriptive variable and function names (`calculateMonthlyRecurringRevenue` not `calcMRR`)
- Keep functions small and focused (ideally under 50 lines)
- Write self-documenting code; add comments only when necessary
- Follow language-specific style guides (PEP 8 for Python, Airbnb for JavaScript)

Embrace Simplicity (KISS Principle):

- Choose the simplest solution that solves the problem
- Avoid over-engineering and premature optimization
- Don't add abstractions until you need them (YAGNI)
- Prefer proven technology over experimental frameworks

Design for Testability:

- Aim for 80%+ test coverage on critical paths
- Write unit tests for individual functions
- Include integration tests for component interactions
- Use dependency injection to make components testable
- Test edge cases and error conditions, not just happy paths

1.3 Technical Stack

Backend: Python 3.11+ (FastAPI), Node.js 20+ for real-time features

Frontend: React 18+ with TypeScript, Tailwind CSS

Database: PostgreSQL 15+, Redis 7+ (cache)

Infrastructure: AWS (EKS/Kubernetes), GitHub Actions, ArgoCD

Monitoring: Datadog, Sentry, OpenTelemetry

Adding new technology requires an RFC (Request for Comments) document and team consensus.

1.4 Security Requirements

All engineers are responsible for writing secure code:

- Use OAuth 2.0/OIDC for authentication
- Implement role-based access control (RBAC) on all endpoints
- Encrypt sensitive data at rest (AES-256) and in transit (TLS 1.3)
- Hash passwords with bcrypt (minimum 12 rounds)
- Sanitize all user input to prevent injection attacks
- Never log sensitive information (passwords, tokens, PII)
- Rate limit public endpoints (100 req/min per IP)

Security review required for all authentication/authorization changes.

1.5 Performance Targets

- API response time: p95 < 200ms, p99 < 500ms
- Page load time: < 2 seconds on 3G connection
- Database query time: < 100ms for 95% of queries
- Design stateless services for horizontal scaling
- Use caching aggressively (Redis)
- Queue non-urgent work (emails, analytics)

1.6 Documentation Standards

Required Documentation:

- README.md in every repository with setup instructions
- API documentation via OpenAPI/Swagger specs
- Architecture Decision Records (ADRs) for significant decisions
- Inline comments for complex business logic
- Migration guides for breaking changes

---

2. Code Reviews

2.1 Review Requirements

All code changes must be **reviewed by at least one other engineer** before being merged into the main branch. No exceptions, even for urgent hotfixes.

Reviewer Assignment:

- **Small PRs** (<200 lines): 1 reviewer
- **Medium PRs** (200-500 lines): 2 reviewers
- **Large PRs** (500+ lines): 2+ reviewers (consider breaking up)
- **Security-sensitive changes**: Security champion must review

2.2 Pull Request Process

1. Create Pull Request:

- Use descriptive title: `[Component] Brief description`
- Complete PR template (What, Why, How, Testing, Screenshots)

- Link related Jira ticket(s)

- Ensure all CI checks pass before requesting review

2. Automated Checks (must pass):

- [ ] Unit tests (90%+ coverage)

- [ ] Integration tests

- [ ] Linting (Pylint, ESLint)

- [ ] Type checking (mypy, TypeScript)

- [ ] Security scanning (Snyk)

3. Review Timeline:

- Reviewers respond within **4 business hours**

- Complete review within **24 hours** (normal priority)

- Urgent PRs: **2 hours** (Slack reviewer directly)

4. Merge:

- All reviewers must approve

- All comments resolved

- All CI checks green

- Use "Squash and Merge" to keep history clean

2.3 What to Review

Correctness: Does it work? Edge cases handled? Error handling appropriate?

Design: Fits architecture? Overly complex? Better approaches available?

Readability: Clear names? Logical structure? Would new team member understand?

Testing: Adequate tests? Edge cases covered? Tests actually test the right things?

Performance: Obvious issues? N+1 queries? Caching appropriate?

Security: Input validation? Authentication/authorization? SQL injection or XSS risks?

2.4 Feedback Guidelines

Giving Feedback:

- Be kind and constructive; assume positive intent
- Be specific with examples
- Distinguish severity:
  - □ **BLOCKING**: Bugs, security issues (must fix)
  - □ **SUGGESTION**: Improvements (nice to have)
  - 🔍 **NIT**: Minor style preferences (author's choice)
- Explain the "why" behind feedback
- Praise good work explicitly

Example:

```
✗ "This is confusing"
```

✅ "This function name doesn't indicate it has side effects. Consider

renaming to `create_and_send_invoice` to make it explicit."

```

**Receiving Feedback**:

- Don't take it personally

- Respond to every comment

- Ask clarifying questions if confused

- Push back respectfully if you disagree (with reasoning)

- Thank reviewers for catching issues

### 2.5 PR Size Guidelines

- **Small** (<200 lines): ✅ Ideal - fast to review, low risk

- **Medium** (200-500 lines): ⚠ Acceptable - consider breaking up

- **Large** (500-1000 lines): □ Avoid - difficult to review thoroughly

- **Extra Large** (>1000 lines): ⛔ Must break up (except auto-generated code)

For large features, create incremental PRs against a feature branch, then merge feature branch to main.

---

## 3. Deployment Process
```

3.1 CI/CD Pipeline

NimbusHR uses **continuous integration and continuous deployment (CI/CD)**. **Production deployments occur automatically after all checks pass.**

Pipeline Stages:

1. **Build**: Compile code, install dependencies
2. **Test**: Run unit, integration, and E2E tests
3. **Security Scan**: Check for vulnerabilities
4. **Deploy to Staging**: Automatic deployment
5. **Smoke Tests**: Critical path validation
6. **Deploy to Production**: Automatic if all checks pass
7. **Post-Deploy Verification**: Health checks and monitoring

Timeline: Merge to `main` → Production in **15-20 minutes**

3.2 Deployment Strategies

Rolling Deployment (default for backend):

- 25% of pods updated at a time
- Health checks between batches
- Automatic rollback if health checks fail

Blue-Green Deployment (database schema changes):

- New version deployed alongside old
- Traffic switched after validation

- Old version kept for quick rollback

Canary Deployment (high-risk changes):

- 5% traffic to new version initially
- Gradually increase to 100% over 2 hours
- Monitor error rates and latency

3.3 Environments

Development (local): Personal development on engineer's machine

Staging (staging.nimbushr.io):

- Mirrors production environment
- Automated deployments on merge to `main`
- Used for QA and final testing
- Refreshed with production data weekly (anonymized)

Production (app.nimbushr.io):

- Live customer environment
- Automatic deployments after staging validation
- 99.9% uptime SLA
- Multi-region (us-east-1 primary, us-west-2 DR)

3.4 Deployment Controls

Feature Flags: Use LaunchDarkly for gradual rollouts

- Enable features for internal users first
- Roll out to 10% → 50% → 100% of customers
- Kill switch for immediate feature disable

Deployment Windows:

- **Preferred**: Tuesday-Thursday, 10 AM - 4 PM EST
- **Avoid**: Fridays, weekends, holidays, end-of-quarter
- **Freeze**: Last week of quarter (exception: critical fixes)

Emergency Deployments: Require approval from on-call engineer and engineering manager

3.5 Database Migrations

Migration Process:

1. Write backward-compatible migration (additive changes only)
2. Deploy application code that works with old and new schema
3. Run migration during maintenance window (Sunday 2-4 AM UTC)
4. Deploy code that requires new schema
5. Remove old schema after 1 week

Destructive changes (dropping columns/tables):

- Requires 2-week deprecation period
- Must notify all teams
- Verify no dependencies via code search

3.6 Rollback Procedures

Automatic Rollback: Triggered if:

- Health checks fail (>5% error rate)

- Response time increases >50%

- Deployment doesn't complete in 30 minutes

Manual Rollback:

1. Notify #engineering Slack channel

2. Revert via ArgoCD UI or `git revert` + push

3. File incident report

4. Schedule post-mortem

Rollback Time: Target < 10 minutes for critical issues

3.7 Monitoring & Alerts

Deployment Dashboards:

- Real-time deployment status in ArgoCD

- Error rates and latency in Datadog

- Application logs in CloudWatch

Alerts (sent to #alerts Slack):

- Deployment failures

- Error rate > 1%
- p95 latency > 500ms
- Database connection pool exhaustion
- Disk usage > 80%

---

4. Incident Management

4.1 Incident Classification

Severity Levels:

SEV-1 (Critical): Complete service outage or data loss
- Customer-facing features completely unavailable
- Data corruption or unauthorized access
- Response time: Immediate
- Example: Database offline, authentication broken

SEV-2 (High): Major functionality degraded
- Core features significantly impaired but partially functional
- Affecting >20% of customers
- Response time: Within 30 minutes
- Example: Slow page loads, integration failures

SEV-3 (Medium): Minor functionality impaired

- Non-critical features affected
- Workarounds available
- Response time: Within 4 hours
- Example: Report generation failing, email delays

SEV-4 (Low): Cosmetic or minimal impact

- UI bugs, typos, minor performance issues
- Affecting <5% of users
- Response time: Next business day
- Example: Misaligned button, incorrect label

4.2 Incident Response Playbook

In the event of a production incident, engineers must follow the incident response playbook:

1. Detect & Alert (0-5 minutes):

- Automated alerts trigger via Datadog/Sentry/PagerDuty
- Customer reports via support channels
- Internal team member notices issue

2. Acknowledge (0-5 minutes):

- On-call engineer acknowledges alert in PagerDuty
- Create incident in Slack: `/incident create` in #incidents channel
- Post initial status in #general: "Investigating [issue description]"

3. Assess & Classify (5-15 minutes):

- Determine severity level (SEV-1 through SEV-4)
- Identify scope: which customers/features affected
- Check recent deployments, code changes, infrastructure events
- Assign Incident Commander (IC) if SEV-1 or SEV-2

4. Assemble Team (10-20 minutes):

For SEV-1/SEV-2, page additional team members:

- **Incident Commander**: Coordinates response (usually on-call engineer)
- **Technical Lead**: Diagnoses and fixes issue
- **Communications Lead**: Updates stakeholders
- **Customer Support Liaison**: Handles customer inquiries

5. Communicate (Ongoing):

- Update #incidents Slack channel every 15-30 minutes
- Post status page update at status.nimbushr.com
- Notify affected customers via email (CS team)
- Internal updates in #engineering

6. Diagnose & Mitigate (20+ minutes):

- Check logs in CloudWatch/Datadog
- Review recent deployments and code changes
- Query database for errors or locks
- Check infrastructure health (CPU, memory, disk)

Mitigation Options:

- Rollback recent deployment
- Scale up infrastructure resources
- Enable feature flag to disable problematic feature
- Apply hotfix (still requires code review)
- Database query optimization or index creation

7. Resolve (Variable):

- Verify issue is fully resolved
- Monitor for 30 minutes to ensure stability
- Mark incident as resolved in PagerDuty
- Post resolution message: "✅ RESOLVED: [brief explanation]"
- Thank team members for their help

8. Post-Mortem (Within 48 hours):

See section 4.3 below

4.3 Post-Mortem Process

Within 48 hours of resolution, engineers must document post-mortem findings:

Post-Mortem Template (Notion document):

Incident Summary:

- Date/time: When incident occurred and resolved
- Severity: SEV-1, SEV-2, etc.
- Duration: Total downtime or degradation period
- Impact: Number of customers affected, revenue impact
- Root cause: One-sentence summary

Timeline (detailed chronology):

- 10:23 AM - Deployment to production completed
- 10:31 AM - Alert triggered for increased error rate
- 10:35 AM - On-call engineer acknowledged, began investigation
- 10:45 AM - Root cause identified (database connection pool exhausted)
- 11:00 AM - Mitigation applied (scaled up connection pool)
- 11:15 AM - Verified resolution, monitoring for stability
- 11:45 AM - Incident marked resolved

Root Cause Analysis:

- What happened and why (technical details)
- Why existing safeguards didn't prevent it
- Contributing factors

Impact Assessment:

- Customers affected: Count and percentage
- Features impacted: List of affected functionality
- Duration of impact: For each severity level
- Business impact: Lost revenue, support tickets, customer churn risk

What Went Well:

- Fast detection via automated monitoring
- Clear incident response process
- Effective team coordination

What Went Poorly:

- Delay in escalating to SEV-1
- Unclear ownership initially
- Lack of runbook for this scenario

Action Items (with owners and due dates):

- [ ] Add connection pool monitoring alert (@jane, Feb 15)
- [ ] Create runbook for database scaling (@mike, Feb 18)
- [ ] Implement circuit breaker pattern (@sarah, Feb 28)
- [ ] Add load testing to staging environment (@team, Mar 15)
- [ ] Post-mortem review meeting scheduled (@manager, Feb 10)

Post-Mortem Review:

- Schedule review meeting within 1 week
- Share post-mortem doc with entire engineering team
- Discuss lessons learned and action items
- No blame culture - focus on systems and processes
- Track action items to completion

4.4 On-Call Responsibilities

On-Call Rotation:

- Weekly rotation (Monday-Monday)
- Scheduled in PagerDuty
- Primary and secondary on-call engineers
- Compensation: $500/week on-call stipend + time-off-in-lieu for hours worked

On-Call Duties:

- Respond to PagerDuty alerts within 15 minutes
- Monitor #alerts Slack channel
- Triage and resolve incidents
- Escalate when needed
- Document incidents in post-mortems
- Handoff notes to next on-call engineer

On-Call Preparation:

- Review runbooks before shift
- Ensure laptop, phone, and VPN access working
- Have contact info for escalation
- Be available and able to respond (not traveling, drinking, etc.)

4.5 Common Incident Scenarios & Runbooks

Database Connection Pool Exhausted:

1. Check connection count: `SELECT count(*) FROM pg_stat_activity;`

2. Kill idle connections: `SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state = 'idle';`

3. Scale up connection pool in application config

4. Restart application pods if needed

High Memory Usage:

1. Identify memory-hogging pods in Kubernetes dashboard

2. Check for memory leaks in recent code changes

3. Restart affected pods: `kubectl rollout restart deployment/<name>`

4. Scale horizontally if needed: `kubectl scale deployment/<name> --replicas=10`

API Rate Limit Exceeded:

1. Identify source IP in nginx logs

2. Check if legitimate traffic spike or attack

3. Temporarily increase rate limit or block IP

4. Contact customer if legitimate usage

Third-Party Integration Failure:

1. Check integration status page (Gusto, ADP, etc.)

2. Review error logs for API failures

3. Enable circuit breaker to fail gracefully

4. Queue requests for retry when service recovers

5. Notify customers via status page

Full Disk:

1. Identify what's consuming space: `df -h` and `du -sh /*`

2. Clear old logs: `find /var/log -name "*.log" -mtime +7 -delete`

3. Increase disk size in AWS console

4. Set up disk usage alerts if missing

4.6 Incident Communication Templates

Initial Status Update:

```

 We're investigating reports of [issue description].

Our team is actively working on a resolution.

We'll provide updates every 30 minutes.

Status: https://status.nimbushr.com

```

Progress Update:

```

UPDATE: We've identified the root cause as [brief explanation].

Our team is implementing a fix.

Estimated resolution: [time].

Affected features: [list].

```

Resolution Notice:

```
```

✅ RESOLVED: The issue with [description] has been fully resolved

as of [time]. All systems are operating normally.

Root cause: [brief explanation].

We apologize for any inconvenience.

```

---

**Questions or Suggestions?**

Contact the Engineering Team at engineering@nimbushr.com or post in #engineering on Slack.

**Last Updated**: January 2025 | **Next Review**: July 2025
```